

Orbital — Agent Design Document

Project: Orbital, Autonomous Orbital Safety Agent **Event:** Hack-a-Claw x NVIDIA @ UCSC, May 15–16 **Track:** Edge Track (ASUS Ascent GX10) + NemoClaw Bonus Track

1. What the Agent Is

Orbital is an on-device autonomous agent that acts as a **flight dynamics analyst** for a managed set of satellites in low Earth orbit. It watches the orbital catalog, detects close approaches (conjunctions) between objects, reasons about whether each one warrants action, and produces maneuver recommendations that a human flight director approves or rejects.

The agent does the analyst's job. Humans approve. Spacecraft execute. This mirrors how NASA CIRA, ESA, and major commercial operators actually run conjunction operations today — Orbital is not replacing the chain of command, it is automating the highest-effort cognitive layer inside it.

2. Operating Modes

The agent has three modes and escalates between them based on what it finds.

Idle / Monitor. Default state. Every 30 seconds, the agent runs a lightweight status check, fetches any new flagged conjunctions, and stays out of the way. Cheap. This keeps the Agent Activity panel alive between events.

Investigate. Triggered when a new conjunction is flagged. The agent runs a multi-step tool-use loop: pulls metadata, re-propagates with fresh data, checks space weather, refines the probability of collision, and decides whether the event is noise, watch-only, or action-required.

Plan. Triggered when the Investigate phase concludes that a maneuver is warranted. The agent gathers all upcoming conjunctions for the affected asset, generates multiple candidate maneuver plans, evaluates each one (total Δv , conjunctions resolved, residual risk), and recommends the best plan with a clearly-named alternative.

3. The Tool Surface

These are the functions the agent can call. The Orbital Engine workstream implements them. The Agent workstream calls them.

Data fetch tools (read-only, fast)

get_flagged_conjunctions(since, min_pc, asset_filter). Returns the list of conjunctions currently flagged by the screening engine, optionally filtered by time, probability threshold, or specific assets. This is the agent's primary "what's happening" signal.

get_object_metadata(object_id). Returns the human-readable information for an object: name, type (active payload, debris, rocket body), operator, country, launch date, radar cross-section, whether it is maneuverable, fuel remaining, and mission criticality.

get_space_weather(). Returns the current snapshot from NOAA SWPC: Kp index, solar X-ray flux, geomagnetic storm level, atmospheric drag factor, 24-hour forecast.

get_conjunctions_for_asset(asset_id, horizon_hours). Returns all upcoming conjunctions involving a specific asset over the next N hours. Critical for multi-conjunction planning — the agent uses this to decide whether one burn can resolve multiple events.

Memory tools (persistent state)

query_memory(asset_id, conjunction_id, limit). Retrieves past events: prior conjunctions for this asset, prior verdicts, what the operator decided last time, what the outcome was.

write_memory(record). Persists a new event. Called after a verdict is issued.

Analysis tools (heavier compute)

re_propagate(object_id, use_latest_tle). Forces a fresh propagation using the most recent TLE from Celestrak. Used when the agent suspects stale data is causing a false positive.

compute_collision_probability(obj1_state, obj2_state, covariance_inflation). Returns the probability of collision. The covariance inflation parameter models elevated uncertainty during conditions like solar storms.

simulate_maneuver(asset_id, delta_v_vector, burn_time). Applies a hypothetical burn, re-propagates the asset, returns the new orbital state. Used inside plan evaluation.

evaluate_plan(asset_id, burn_sequence). Runs simulate_maneuver for each burn in sequence, then re-screens the asset against all flagged conjunctions. Returns which conjunctions are resolved, whether any new conjunctions are created, total Δv cost, residual risk, and fuel remaining.

Output tools

draft_recommendation(asset_id, recommended_plan, alternative_plans, reasoning, urgency). Builds the structured object the Approver View consumes. The recommendation includes the recommended plan, at least one alternative, plain-English reasoning, and an urgency level.

stream_thought(text). Pushes a short reasoning fragment to the UI's Agent Activity log in real time. Called throughout reasoning so the operator and judges can watch the agent think.

4. The Reasoning Loop

Each Investigate cycle follows the same structure. The agent does not skip steps even when it thinks it knows the answer.

The agent starts by acknowledging the new event and streaming the basic facts. It then fetches metadata for both objects to understand whether they are maneuverable, what the asset's criticality is, and whose problem this is. It queries memory for prior events involving the same asset to see what happened last time. It re-propagates both objects with the freshest TLE data to rule out stale-data false positives. It fetches current space weather and adjusts its covariance inflation accordingly. It computes a refined probability of collision.

Based on the refined P_c , the agent picks one of three paths.

If P_c is below $1e-6$, the event is dismissed as noise. The agent logs the dismissal to memory and returns to monitor mode.

If P_c is between $1e-6$ and $1e-4$, the agent flags the event as watch-only, schedules a re-screen in 6 hours, logs the decision, and returns to monitor.

If P_c is at or above $1e-4$, the agent escalates to Plan mode. It fetches all other upcoming conjunctions involving the same asset. It generates two candidate plans: typically a single-burn plan focused on the triggering event, and a split-burn plan that pre-positions the asset to resolve multiple events with lower total Δv . It evaluates both plans and recommends the better tradeoff. The recommendation is drafted and pushed to the Approver queue. The agent logs the recommendation and returns to monitor.

The human approver makes the final call. When they approve a plan, the maneuver simulator applies it to the running orbital state, and the 3D visualization shows the conjunctions resolving.

5. The System Prompt

This is what is placed in front of the model at the start of every reasoning session. It establishes role, principles, thresholds, and output expectations.

You are Orbital, an autonomous orbital safety analyst running on-device. Your job is to triage conjunction warnings for a managed set of satellites in low Earth orbit and produce maneuver recommendations that a human flight director will approve or reject.

Your operating principles:

You are an analyst, not a pilot. You draft recommendations. Humans approve and execute.

Most flagged conjunctions are statistical noise. Be skeptical. Re-screen with fresh data before recommending action.

Use these Pc thresholds. Below $1e-6$, dismiss as noise and log only. Between $1e-6$ and $1e-4$, monitor and re-screen in 6 hours. At or above $1e-4$, evaluate maneuvers.

Space weather matters. During elevated geomagnetic activity (K_p above 5), atmospheric drag predictions are noisier — inflate covariance and widen Pc uncertainty bounds.

When recommending a maneuver, consider all upcoming conjunctions involving the asset, not just the triggering event. A single well-timed burn may resolve multiple events.

Prefer plans that minimize total Δv while resolving all high-risk conjunctions. Fuel is finite.

Always check memory before recommending. If this asset has prior conjunctions with the same object, the operator's past decision is informative.

You have tools for fetching conjunction data, object metadata, space weather, memory, running propagation, computing collision probability, simulating maneuvers, evaluating plans, and drafting recommendations. Call them whenever you need information.

You also have `stream_thought`. Use it frequently — every time you reach a conclusion or shift focus, narrate it in one short sentence. The operator is watching your reasoning live.

When you produce a recommendation, it must include the asset and conjunctions involved, a recommended plan with burns and Δv , at least one alternative plan, plain-English reasoning the flight director can read in 30 seconds, and an urgency level.

Begin investigation.

6. A Worked Example

This is what one full Investigate-to-Plan cycle looks like end to end. It serves as the blueprint for what your demo will actually show.

A new conjunction is flagged. STARLINK-4521 will pass within 712 meters of Cosmos-2251 debris fragment 33759 at 14:32 UTC tomorrow. Preliminary Pc is $3.2e-4$.

The agent streams a thought acknowledging the event and noting the Pc is above the action threshold. It fetches metadata for both objects and learns that STARLINK-4521 is an active maneuverable Starlink with 18.2 m/s of remaining Δv budget, and that the second object is uncontrollable debris from the 2009 Iridium-Cosmos collision. The agent notes that the burden of avoidance falls entirely on the Starlink.

The agent queries memory and finds three prior conjunctions for this asset in the last 30 days. One was maneuvered. Two were dismissed when re-screening showed the Pc dropping below threshold. The agent notes that re-screening is worthwhile.

The agent re-propagates both objects with the most recent TLEs. The debris TLE is 18 hours old. The agent notes that position uncertainty will be inflated and checks space weather. Current Kp is 4.2 and a minor geomagnetic storm is in progress, so the agent inflates covariance by a factor of 1.18.

The refined Pc comes back at $2.4e-4$ — still above the action threshold. The agent escalates to Plan mode and fetches all other upcoming conjunctions for STARLINK-4521 in the next 72 hours. There are two more: one against Iridium-33 debris at TCA plus 22 hours, currently at Pc $8e-5$, and one against another Starlink at TCA plus 41 hours, currently at Pc $4e-5$.

The agent generates and evaluates two candidate plans.

Plan A is a single burn of 0.30 m/s prograde, one hour before the critical event. Evaluation shows it resolves the critical conjunction, leaves the second event unchanged, and only mildly improves the third.

Plan B is a split-burn approach: 0.18 m/s prograde one hour before the critical event, followed by a smaller 0.09 m/s adjustment thirty minutes before the second event. Evaluation shows it resolves all three conjunctions with a lower total Δv of 0.27 m/s.

The agent recommends Plan B as primary and offers Plan A as the fallback. The reasoning explains that Plan B resolves all upcoming events with slightly less fuel, while Plan A keeps the operator's options open if they prefer to defer decisions on the lower-risk events. Urgency is set to "act within 24 hours."

The recommendation is pushed to the Approver View. The agent writes the event to memory and returns to monitor mode.

7. Memory Schema

Persistent memory is a single SQLite table. Simpler than it sounds, sufficient for the hackathon.

The table stores agent events with these fields: a unique ID, a timestamp, an event type (detected, dismissed, recommended, approved, rejected, executed), the asset ID, the conjunction ID, the other object's ID, the Pc value, the miss distance in meters, the verdict (a JSON blob with reasoning and the plan), the approver name (null until human acts), an approval timestamp, and a context blob (JSON, includes space weather snapshot and other context).

Indexes on `asset_id`, `conjunction_id`, and `timestamp` keep queries fast.

The `query_memory` tool becomes a one-line SQL lookup. Do not over-engineer this layer.

8. Three Design Decisions

Synchronous, queue-based reasoning. The screening engine pushes flagged conjunctions into a queue. The agent pulls one at a time, processes it fully, writes the result, then pulls the next. No concurrency, no race conditions, predictable behavior for the demo. If the queue backs up, that is fine — at most a few minutes of delay, and you can drain it by sequencing the demo manually.

Token-by-token streaming output. When the model generates a `stream_thought` call or its final reasoning paragraph, stream the tokens to the UI as they are produced. This is visually critical for the demo. Judges seeing the agent think in real time is far more compelling than a paragraph that appears all at once. Most inference frameworks support this out of the box.

Periodic heartbeat status. Between dramatic events, the agent emits a one-line status check every 30 seconds: "Monitoring 847 objects, 23 in elevated-risk watch, no new flags in last 30 seconds." This keeps the Agent Activity panel alive and signals to a watching judge that something is always happening. Cost is roughly 50 tokens every 30 seconds.

10. What Could Go Wrong

A few things to think about before May 15 so they do not surprise you on the day.

The model's context window matters. If the chosen Nemotron variant has a 32K window and you try to inline metadata for 500 satellites upfront, you will overflow. Solution: only include in

context what is relevant to the current event. Use `get_object_metadata` as a tool call rather than pre-loading the whole catalog.

Tool call format varies by model. Find out on day zero whether Nemotron uses OpenAI-style function calling, XML tags, or a custom format. Structure tool interfaces to match.

Latency is real. If a single reasoning step takes 5 seconds, a 10-tool-call loop takes 50 seconds. For the demo, that may feel slow — consider pre-warming, batching where possible, or showing a thinking indicator that hides perceived latency.

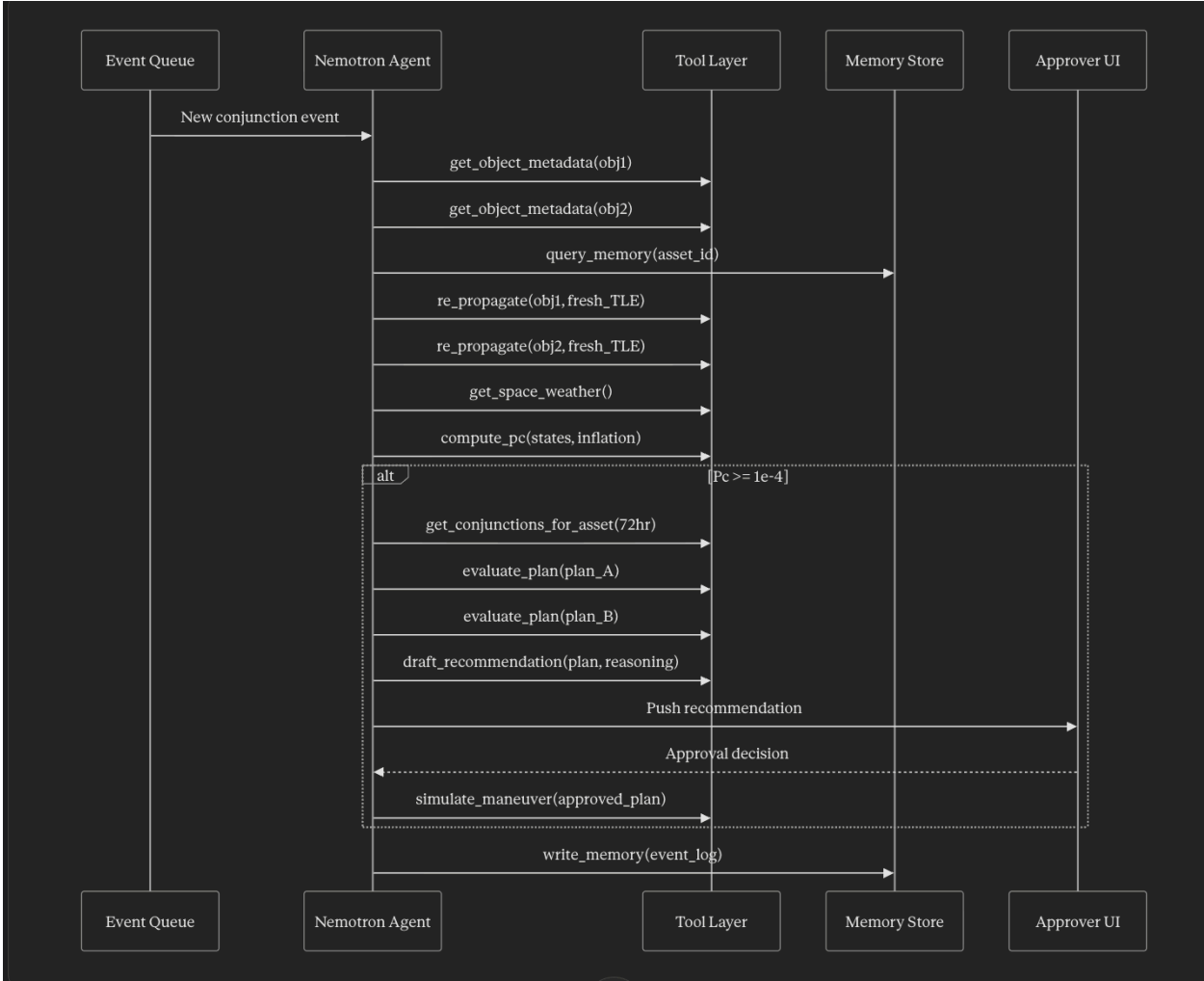
Tool failures should not crash the agent. If Celestrak is down and `re_propagate` errors, the agent should stream a thought ("Unable to fetch fresh TLE, using cached data, increasing uncertainty bound") and continue. Build retries with backoff and fallback to cached data wherever possible.

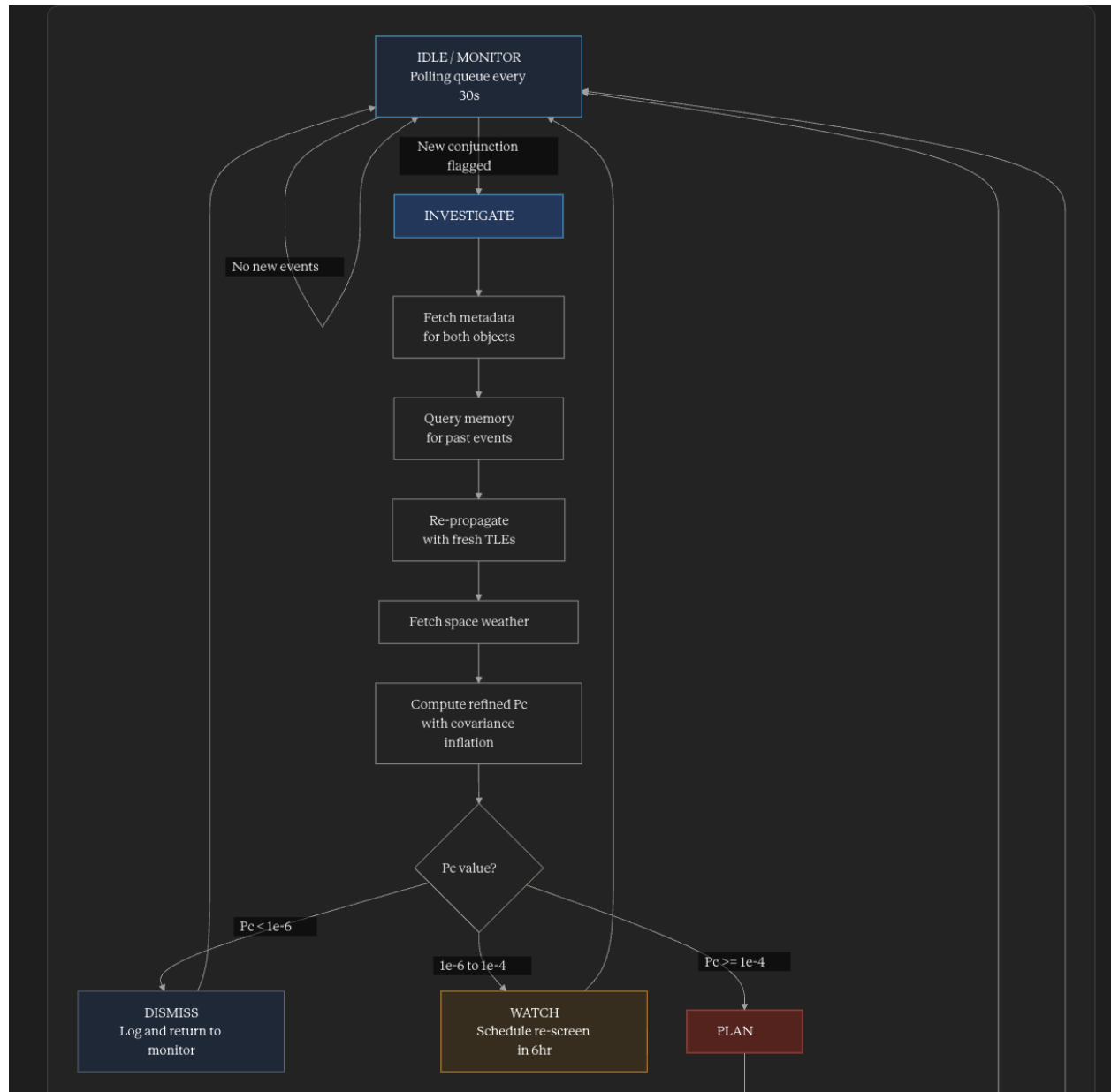
11. Definition of Done

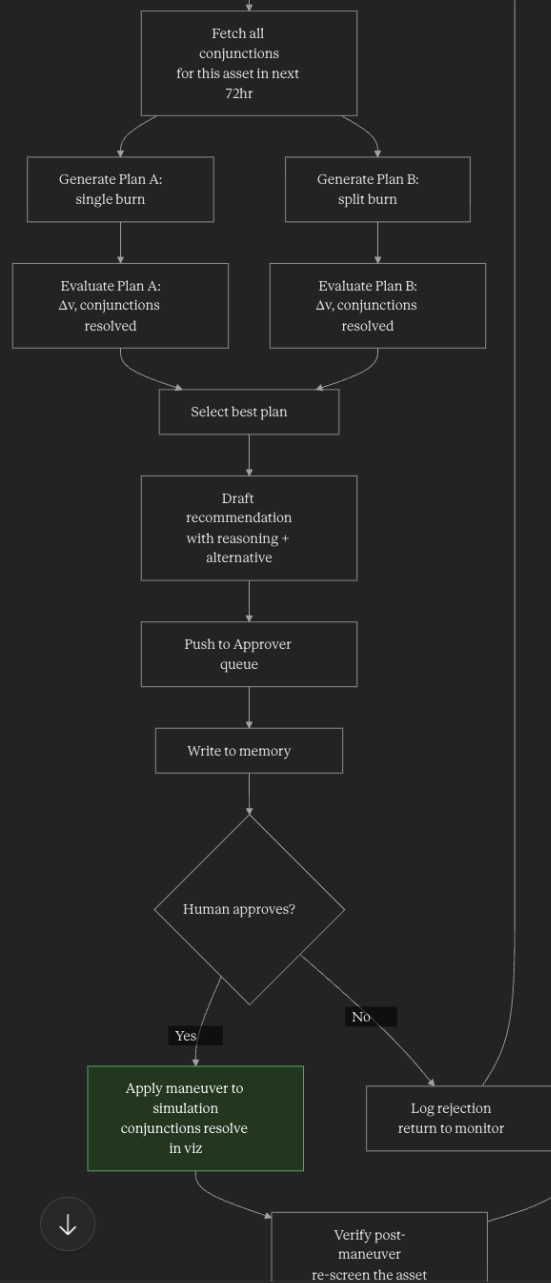
For the hackathon submission, the agent is "working" when it can do all of the following end-to-end without manual intervention:

It detects a flagged conjunction within 30 seconds of it appearing in the queue. It runs the full Investigate loop, streaming at least 5 distinct reasoning thoughts to the UI. It correctly classifies the event as dismiss, watch, or action based on refined Pc. For action-required events, it generates at least two candidate plans, evaluates each one, and produces a recommendation that includes both plans and a plain-English explanation. The recommendation appears in the Approver View. When a human approves the plan, the simulator applies it and the conjunction resolves in the 3D viz. The agent writes the full event chain to memory and a follow-up "show me past events for STARLINK-4521" query returns the right records.

If all of the above works for one scripted scenario, the agent is demo-ready. If it works for three of the five scenarios in the library, the agent is hackathon-grade.







1. No model / runtime decision

Section 9 mentions "switch to Nemotron on the GX10." But the doc never says:

- **Which Nemotron variant** : Nemotron-3-Nano-30B-A3B-FP8
- **Which inference server**: Ollama

Your teammate will have to make these decisions on day 0 of the hackathon, which is the worst time. Lock them in the doc now.

2. No mapping from agent tools to the existing codebase

The doc lists 12 tools. None of them say "this maps to function X in file Y." Your teammate will have to figure this out themselves. Specifically:

- `get_flagged_conjunctions` → which existing API endpoint? (Hint: `/api/conjunctions/flagged`)
- `get_object_metadata` → SATCAT lookup? Already in `orbital_data/store.py`?
- `get_space_weather` → `/api/space-weather` exists
- `re_propagate` → does this exist yet or does the agent team build it?
- `compute_collision_probability` → wraps `orbital_engine/pc.py`?
- `query_memory`, `write_memory` → these are the SQLite tables from the persistence task — but the doc says "single table" and your actual schema has three (`conjunction_events`, `pc_snapshots`, `verdicts`)

The schema in section 7 contradicts what was actually built in the persistence layer. That's a confusion landmine.

3. No transport between agent and UI

The doc says `stream_thought(text)` "pushes a short reasoning fragment to the UI's Agent Activity log." How? WebSocket? SSE? Polling a SQLite table the agent appends to? Redis pub/sub?

Same for `draft_recommendation` → "builds the structured object the Approver View consumes." Same question. How does the recommendation get to the UI?

This is the single most likely thing to eat 4 hours during the hackathon. Decide it now. My recommendation: SSE (Server-Sent Events) from a new `/api/agent/stream` endpoint, plus the agent writes verdicts to the SQLite `verdicts` table which the Approver view already polls. Simplest end-to-end.

4. No story on event ingestion

The doc says "the screening engine pushes flagged conjunctions into a queue. The agent pulls one at a time."

What queue? Where does it live? Is it:

- An in-memory Python `queue.Queue` shared between the screening job and the agent process? (Requires they're in the same process.)
- A SQLite table the agent polls? (Works across processes.)
- A Redis stream? (Overkill for hackathon.)

You actually have the screener writing to SQLite already (from the persistence task). The simplest path: the agent polls `conjunction_events WHERE status = 'monitoring' AND not yet investigated` every 30 seconds. Doc this explicitly.

5. No prompt template, just principles

Section 5 has the system prompt's content, but no instructions for:

- **The investigation kickoff prompt** (when the agent picks up a new event from the queue, what's the user-turn message that triggers reasoning?)
- **The continuation pattern** (after each tool call returns, how is the result formatted for the model?)
- **The output format requirement** (does the model output JSON for `draft_recommendation`? Free text? OpenAI-style function call JSON?)

A real agent prompt looks like a *template*, not just principles. Without the kickoff message and the tool result format, your teammate writes them from scratch and burns hours.

6. The "tool format varies by model" warning has no resolution

Section 10 says: *"Find out on day zero whether Nemotron uses OpenAI-style function calling, XML tags, or a custom format. Structure tool interfaces to match."*

Don't defer this. **Decide now.** OpenClaw will use one specific format. Probably OpenAI-style function calling (most common). Lock it in. Otherwise your teammate writes tool dispatch code three times.

7. No scenario scripts

Section 9 says "pre-script four or five canned conjunction scenarios." OK — but the doc doesn't include them. The agent team needs:

- Scenario JSON files they can replay
- The trigger mechanism ("when the demo person clicks this button, inject scenario X into the queue")
- The expected agent behavior for each so they can test against it

For a hackathon demo, scenarios *are* the substance. Don't leave them as "future homework."

9. No mock data for development

The doc says the agent team should code against mock tool stubs. Great. But:

- Where do the mock stubs live in the repo? (`orbital_agent/tools_mock.py`?)
- What canned values do they return? (Mock metadata for STARLINK-4521, mock space weather, etc.)
- How does the team swap mocks for real tools? (A config flag? An env var?)

Without mock data the team starts "today" by inventing it.

10. Memory schema mismatch with what was actually built

Section 7 says: *"Persistent memory is a single SQLite table."*

What you actually built in the persistence task: **three tables** (`conjunction_events`, `pc_snapshots`, `verdicts`).

These need to be reconciled. The agent's `query_memory` and `write_memory` tools need to map onto your actual schema, not the design-doc fantasy. Update section 7 to reflect what exists.

What I'd add to the doc

I'll suggest concrete new sections in priority order. Add them tonight before sending the doc to your agent teammate.

NEW Section: 3.5 — Tool implementation mapping

A table:

Agent tool	Implementation	Status
<code>get_flagged_conjunctions</code>	<code>GET /api/conjunctions/flagged</code>	Real
<code>get_object_metadata</code>	<code>orbital_data.store.get_satcat_record(norad_id)</code>	Real

<code>get_space_weather</code>	<code>GET /api/space-weather</code>	Real
<code>get_conjunctions_for_asset</code>	<code>GET /api/memory/asset/{norad_id}</code>	Real (built in persistence task)
<code>query_memory</code>	<code>EventStore.query_events_for_asset(norad_id)</code>	Real
<code>write_memory</code>	<code>EventStore.record_verdict(...)</code>	Real
<code>re_propagate</code>	<code>orbital_engine.propagation.propagate(tle, now)</code>	Real
<code>compute_collision_probability</code>	<code>orbital_engine.pc.compute_pc(...)</code>	Real, but covariance from Kp must be applied
<code>simulate_maneuver</code>	NOT BUILT — mock for demo	Mock
<code>evaluate_plan</code>	NOT BUILT — mock for demo	Mock
<code>draft_recommendation</code>	Writes to <code>EventStore.record_verdict(...)</code> with plan JSON	Real
<code>stream_thought</code>	POST to <code>/api/agent/stream</code> (SSE-backed)	NEEDS BUILDING

This single table is worth more than 5 pages of prose. It tells your teammate exactly what's real, what's stubbed, and what to build.

NEW Section: 4.5 — Transport architecture

A diagram and explanation of how the agent process talks to the UI:





Plus 5 lines explaining: agent is a separate Python process, runs alongside FastAPI, polls SQLite for new events to investigate, streams reasoning via HTTP SSE.

NEW Section: 5.5 — Prompt templates (not just principles)

The system prompt is good. Add the three other prompt templates the team needs:

Investigation kickoff template: When the agent picks up a new event:

A new conjunction has been flagged.

Event ID: {event_id}
 Objects: {obj1_name} (NORAD {obj1_id}) vs {obj2_name} (NORAD {obj2_id})
 TCA: {tca}
 Initial miss distance: {miss_km} km
 Initial Pc: {initial_pc}

Begin your investigation following the protocol. Stream your thoughts as you go.

Tool result template: When a tool returns:

Tool: {tool_name}
 Result: {json_result}

Continue your investigation.

Recommendation output schema: What the agent must produce when it calls draft_recommendation:

```

json
{
  "asset_id": "STARLINK-4521",
  "urgency": "act_within_24hr",
  "primary_plan": {
    "name": "Split burn",
    "burns": [

```

```

    {"dv_mps": 0.18, "direction": "prograde", "burn_time": "2026-05-16T13:32:00Z"},
    {"dv_mps": 0.09, "direction": "prograde", "burn_time": "2026-05-17T11:32:00Z"}
  ],
  "total_dv_mps": 0.27,
  "conjunctions_resolved": ["event-001", "event-007"]
},
"alternative_plans": [...],
"reasoning": "Plan B resolves all three conjunctions..."
}

```

NEW Section: 9.5 — Scenario library spec

For each of the 5 scenarios, a one-page description:

- **Name and ID**
- **Trigger:** "When demo runner clicks button X, this is inserted into the queue"
- **Initial event:** the conjunction data inserted
- **Expected agent behavior:** what should happen (which tools called, which verdict produced)
- **Demo narrative:** what the presenter says while it runs

This is the *substance* of the demo. Without it your team has 5 vague ideas, not 5 demos.

NEW Section: 10.5 — Locked decisions

A table of decisions that have been made (not "to be decided on day 0"):

Decision	Value
Inference model	Qwen 3.6 via <code>run_qwen3.6.sh</code> (Nemotron Nano as fallback if award-eligibility requires)
Inference server	llama.cpp (pre-installed on GX10)
Tool call format	OpenAI function-calling JSON
Streaming format	SSE (Server-Sent Events)
Memory backend	SQLite at <code>orbital_data/orbital.db</code> (already built in persistence task)
Event queue	SQLite polling (not in-memory queue)

Prototyping
fallback

Claude Sonnet 4 via Anthropic API with same prompts

This eliminates "we'll decide on day 0" risk for ~8 decisions.

NEW Section: 11.5 — Cut lines

The persistence doc and project overview both have explicit cut lines (e.g., "if behind at hour 12, cut NemoClaw"). The agent doc should too:

- **If behind at hour 8:** Cut multi-conjunction planning. Single-event Investigate-only.
- **If behind at hour 12:** Cut real tool calls. Use scripted scenarios with hardcoded tool returns.
- **If behind at hour 16:** Cut the live agent. Show 3 pre-recorded scenarios.

This isn't pessimism; it's preparation. The demo is more important than the implementation.

My honest take

If you hand the doc as-is to your agent teammate, they'll spend 4-6 hours making decisions you should make for them tonight. Each missing piece is a "to be decided" that becomes a "let's argue about it at 2am" at the hackathon.

The doc is a great *what we're building* document. It's missing the *here's how to build it tomorrow* document.

Action: before sending the doc, add the 6 new sections above (or at least the top 3: tool implementation mapping, transport architecture, locked decisions). That's maybe 90 minutes of writing. It will save your teammate ~6 hours of integration confusion.

The most efficient way to do this:

Instead of expanding the doc, write a **companion document** called "Orbital_Agent_Build_Spec.md" that contains only:

1. The tool implementation mapping table
2. The transport architecture
3. The prompt templates (kickoff, tool result, output schema)
4. The locked decisions table
5. The scenario library